

CONSIDERACIONES GENERALES PARA RESOLVER LOS EJERCICIOS

- No podrá ser corregido ningún programa que no compile.
- No agregar ni quitar funcionalidades.
- No modificar los formatos.
- No utilizar `getch` ya que Linux no lo acepta.
- No utilizar `strcpy`, se deberá crear una función o procedimiento propio.
- Si el examen es escrito debe ser entregado en tinta, con cada hoja correctamente numerada y en el margen superior de cada hoja se debe indicar a qué archivo corresponde dicho código.
- La nota dependerá de la eficiencia.

ÍNDICE

Etapa 1 — Arrays y funciones

1) Array aleatorio: divisibles por 7	2
2) Array aleatorio: posiciones impares	2
3) Array aleatorio: posiciones por 3	2
4) Array aleatorio: posiciones pares	3
5) Array aleatorio: posiciones por 4	3
6) Comparar si dos arrays son iguales	4
7) Array codificado en letras	4
8) Unir impares de dos arrays	5
9) Calcular el CUIT	5
10) Fracciones: suma y simplificación	5

Etapa 2 — Matrices

11) Matriz de balances por bimestre	6
12) Suma de matrices 3 x 4	6
13) Matrices 6 x 6 y diagonales	6
14) Matrices 8 x 8 y promedios	7
15) Matriz 4 x 4 de letras sin repetir	7

Etapa 3 — Dibujos con ciclos

16) Pirámide de caracteres	7
17) Figura con dibujarFigura	8
18) Triángulo con letras del abecedario	8
19) Gráfico de barras de votos	8

Etapa 4 — Punteros y pruebas de escritorio

20) Punteros sobre un vector de 10	9
--	---

21) Punteros sobre un vector de 3	9
22) Prueba de escritorio de una función	10

Etapa 5 — Recursividad

23) Menú: creciente, decreciente y factorial	10
24) Menú: decimal a binario con archivos	10
25) Pasar un número a otra base	10

Etapa 6 — Structs y TDA

26) Struct Libro	11
27) TDA Rectángulo	11
28) TDA Círculo con menú	11
29) TDA Empleado	11
30) TDA Detalle de factura	11
31) TDA Jugador y Equipo	11
32) TDA Jugador de fútbol	11
33) TDA Elemento de la tabla periódica	12
34) TDA Persona y Nacimiento	13
35) TDA Banda y Líder	14
36) Mundial de Qatar y Estadios	15
37) Museo, Director y Obras	15

Etapa 7 — Archivos y búsquedas

38) ventas.txt y búsqueda secuencial	16
39) vehiculos.txt y búsqueda secuencial	16
40) nombres.txt ordenado	16
41) numeros1.txt y búsqueda binaria	16

1)

1. El usuario selecciona la cantidad de valores enteros con los que quiere trabajar, esa cantidad debe ser de 2 cifras e impar.
2. Los elementos deben cargarse aleatoriamente con números negativos de tres cifras.
3. Se debe mostrar el arreglo.
4. Se debe mostrar el arreglo de forma invertida.
5. Mostrar el promedio de los divisibles por 7.

```

----- TEMA 1 -----
---- BENITEZ, Pepe, DNI: 42000000 ---
Ingresar un tamaño impar de dos cifras:
1
Ingresar un tamaño impar de dos cifras:
22
Ingresar un tamaño impar de dos cifras:
13
|| -141 -567 -134 -500 -369 -524 -778 -658 -962 -264 -405 -345 -881 ||
|| -881 -345 -405 -264 -962 -658 -778 -524 -369 -500 -134 -567 -141 ||

PROM: -612.500000

```

2)

1. El usuario selecciona la cantidad de valores enteros con los que quiere trabajar, esa cantidad debe ser mayor a 10, menor que 30 y que no sea divisible ni por 2 ni por 5.
2. Los elementos deben cargarse aleatoriamente con números de dos cifras pares.
3. Se debe mostrar el arreglo.
4. Se debe mostrar solo las posiciones impares del array (1,3,5..etc).
5. Mostrar el mayor de los números menores a 50.

```

----- TEMA 2 -----
---- BENITEZ, Pepe, DNI: 42000000 ---
Ingresar un tamaño de 11 a 29,
que no se pueda dividir por 2 ni por 5
15
Ingresar un tamaño de 11 a 29,
que no se pueda dividir por 2 ni por 5
12
Ingresar un tamaño de 11 a 29,
que no se pueda dividir por 2 ni por 5
9
Ingresar un tamaño de 11 a 29,
que no se pueda dividir por 2 ni por 5
13
|| 28 70 44 56 76 46 74 44 54 94 90 28 64 ||
|| 70 56 46 44 94 28 ||

El mayor de los menores a 50 es: 46

```

3)

1. El usuario selecciona la cantidad de valores enteros con los que quiere trabajar, esa cantidad de 2 cifras y no puede terminar en 7.
2. Los elementos deben cargarse aleatoriamente con números de 2 cifras que pueden ser negativos y positivos.
3. Se debe mostrar el arreglo.
4. Mostrar el arreglo en las posiciones divisibles por 3 (posición 0,3, 6, 9, etc).

5. Mostrar el máximo de los impares negativos.

```

----- TEMA 3 -----
---- BENITEZ, Pepe, DNI: 42000000 ---
Ingresar un tamaño de 2 cifras que no puede terminar en 7
17
Ingresar un tamaño de 2 cifras que no puede terminar en 7
6
Ingresar un tamaño de 2 cifras que no puede terminar en 7
14
|| -49 -97 -49 41 28 52 -14 10 22 89 15 -16 -20 13  ||
|| -49 41 -14 89 -20  ||
El mayor de los impares negativos: -14

```

4)

1. El usuario selecciona la cantidad de valores enteros con los que quiere trabajar, esa cantidad debe ser mayor a 15, menor que 40 y que no sea divisible ni por 3 ni por 7 ni por 11.
2. Los elementos deben cargarse aleatoriamente con números de dos cifras múltiplos de 3.
3. Se debe mostrar el arreglo.
4. Se debe mostrar solo las posiciones pares del array (0.2.4..etc).
5. Mostrar el mayor de los números menores a 70.
6. Buscar el número 81 y mostrar el resultado de la búsqueda.

```

----- TEMA 4 -----
---- BENITEZ, Pepe, DNI: 42000000 ---
Ingresar un tamaño de 16 a 39,
que no se pueda dividir por 3 ni por 7 ni por 11.
5
Ingresar un tamaño de 16 a 39,
que no se pueda dividir por 3 ni por 7 ni por 11.
18
Ingresar un tamaño de 16 a 39,
que no se pueda dividir por 3 ni por 7 ni por 11.
28
Ingresar un tamaño de 16 a 39,
que no se pueda dividir por 3 ni por 7 ni por 11.
33
Ingresar un tamaño de 16 a 39,
que no se pueda dividir por 3 ni por 7 ni por 11.
17
|| 84 63 36 78 42 60 42 51 33 51 39 15 33 63 21 42 36  ||
|| 84 36 42 42 33 39 33 21 36  ||
El mayor de los menores a 70 es: 63
No se encontro el numero: 81
Process returned 0 (0x0)   execution time : 13.550 s
Press any key to continue.

```

5)

1. El usuario selecciona la cantidad de valores enteros con los que quiere trabajar, esa cantidad de 2 cifras y no puede terminar en 3 ni en 5.
2. Los elementos deben cargarse aleatoriamente con números de 3 cifras que pueden ser negativos y positivos.
3. Se debe mostrar el arreglo.
4. Mostrar el arreglo en las posiciones divisibles por 4 (posición 0,4,8,12, etc).
5. Mostrar el máximo de los impares negativos.
6. Buscar el número -120.

```

----- TEMA 5 -----
---- BENITEZ, Pepe, DNI: 42000000 ---
Ingresar un tamaño de 2 cifras que no puede terminar en 3 ni 5
15
Ingresar un tamaño de 2 cifras que no puede terminar en 3 ni 5
13
Ingresar un tamaño de 2 cifras que no puede terminar en 3 ni 5
8
Ingresar un tamaño de 2 cifras que no puede terminar en 3 ni 5
14
|| -439 442 -943 -489 225 203 102 813 -483 -349 -372 817 106 -866 ||
|| -439 225 -483 106 ||

El mayor de los impares negativos: -349
No se encontro el numero: -120
Process returned 0 (0x0)   execution time : 4.074 s
Press any key to continue.

```

- 6) Crear un programa que permita cargar dos arrays unidimensionales de enteros, con números aleatorios negativos mayores a -100. Ambos arrays tienen el mismo tamaño, el mismo es un número aleatorio impar de dos cifras. Una vez cargados los arrays se pide crear un procedimiento que diga si ambos arrays son iguales o no. Dos arrays son iguales si tienen exactamente los mismos números, sin importar la posición.
Ejemplo para comprender la igualdad: (1,2,5) es igual que (1,2,5) y que (2,1,5), etc.

- 7) Utilizando el concepto de funciones y procedimientos y fundamentalmente el correcto uso de arrays con funciones, generar un programa que haga lo mismo que la siguiente captura:

<pre> ----- RECUPERATORIO TEMA 2 ----- ----- PEPE BENITEZ, DNI: 1111111 ----- -----> ARRAY: (19 22 10 5 11 16 2 11 7 16 2 17 6 15 3) -----> ARRAY CODIFICADO: (s v j e k p b k g p b q f o c) -----> ARRAY M en adelante: (s v p p q o) </pre>	Array de 15 posiciones en cada posición un aleatorio del 1 al 25. Se crea y guarda un nuevo array, que codifica al array anterior, transformando cada número en la letra que le corresponde del abecedario. Es decir el 1 lo transforma a A, el 3 a C, el 5 a E, etc. Se crea y se guarda un 3er array donde se guardan solo las letras que esten despues de la M.
--	---

- 8) Utilizando el concepto de funciones y procedimientos y fundamentalmente el correcto uso de arrays con funciones, generar un programa que haga lo mismo que la siguiente captura:

```

----- RECUPERATORIO TEMA 3 -----
----- PEPE BENITEZ, DNI: 1111111 -----

-----> ARRAY:
( 7 18 19 6 1 12 )

-----> ARRAY:
( 15 2 7 22 11 17 15 7 17 16 )

-----> ARRAY:
( 19 15 11 17 15 17 )

```

Crear un array de un tamaño aleatorio, menor a 15 elementos, los elementos deben ser positivos.

Crear otro array de tamaño aleatorio menor a 15, y de numeros positivos.

Crear un nuevo array con la cantidad necesaria de elementos para poder almacenar todos los impares de dos cifras del array 1 y del array 2 ...

- 9) Utilizando el concepto de funciones y procedimientos y fundamentalmente el correcto uso de arrays con funciones, generar un programa que haga lo siguiente:

- Crear un array para almacenar el CUIT de una persona.
- El CUIT consta de 11 números enteros. Los dos primeros son 20, 27 o 30 (20 Masculinos, 27 femeninos, 30 sociedades), los que siguen el DNI de la persona y el ultimo es una operación matemática.
- Pedir por teclado una S, M, o F para definir si se trata de una empresa o un ser humano. (Validarlo)
- Cargar aleatoriamente un DNI en el array del cuit.
- Calcular el último número del CUIT y mostrarlo terminado.

Así se calcula, para el: 20 - 12345678 - R

Suma = $2*5 + 0*4 + 1*3 + 2*2 + 3*7 + 4*6 + 5*5 + 6*4 + 7*3 + 8*2$

Resto = suma % 11

El resto determina el último número:

- Si es cero el último número es cero.
- Si es uno, el último número es 9 para hombres, y 4 para mujeres (Cuando el resto es 1 los dos primeros dígitos pasan a ser 23)
- Si el resto no es ni 0 ni 1, el número es 11 - resto

- 10) Utilizando el concepto de funciones y procedimientos y fundamentalmente el correcto uso de arrays con funciones, generar un programa que haga lo mismo que la siguiente captura:

```

----- RECUPERATORIO TEMA 1 -----
----- PEPE BENITEZ, DNI: 1111111 -----

Fraccion: 18/12
Fraccion: 6/98
Resultado suma:-----
Fraccion: 572/688
Resultado suma simplificado:-----
Fraccion: 143/158

```

Crear dos fracciones, cualquiera, con numeradores y denominadores positivos de una o dos cifras

Mostrar la suma entre esas dos fracciones y guardar el resultado

Simplificar el resultado de la suma anterior...

- 11) Se tiene un total de 5 empresas, donde se han guardado los balances de estas por bimestres (6, seis bimestres). Se pide cargar una matriz para almacenar todos estos datos, cargar por teclado. Luego mostrar dicha matriz. Una vez llena la matriz, se pide calcular el total ganado por cada empresa en el año, y el total de ganancias por bimestre. Estos totales deben ser guardados en vectores. Todos los totales deben ser mostrados por pantalla.

Las funciones y procedimientos necesarios son las dadas a continuación, deben ser respetadas:

- void cargarMatriz(int matriz[5][6]);
- void mostrarMatriz(int matriz[5][6]);
- void calcularTotalEmpresas(int matriz[5][6], int totalEmpresas[5]);
- void calcularTotalBimestre(int matriz[5][6], int totalBimestre[6]);
- void mostrarTotalEmpresas(int totalEmpresas[5]);
- void mostrarTotalBimestre(int totalBimestre[6]);
- int maximoMatriz(int matriz[5][6]);

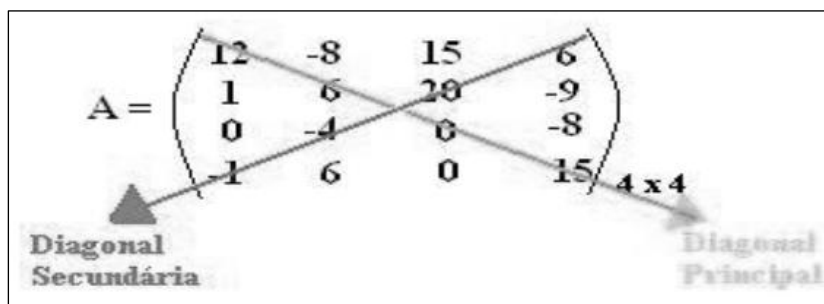
Por último, se pide identificar la máxima ganancia. Todo debe ser llamado desde el **main** en el orden correcto las veces que sea necesario y creando los arreglos que sean necesarios de forma local en el **main**.

- 12) Se tienen dos matrices de 3x4, **matriz1** y **matriz2**, las cuales describen las ventas de dos sucursales de una empresa. En las matrices aparecen las ventas de 3 productos (columnas 4) en cada estación del año (filas 4). Se pide crear una **matriz3**, que sea la suma de estas dos matrices. Una vez sumadas las matrices calcular el total de ventas por producto y el total de ventas por estación. Calcular el promedio de ventas totales de la **matriz3**. No pueden faltar estas funciones/procedimientos:

- void cargarMatriz(int m [3][4]);
- void sumarMatriz(int m1[3][4], int m2[3][4], int m3[3][4]);
- void ventasPorProducto(int m3[3][4], int vector[3]);
- void ventasPorEstacion(int m3[3][4], int vector[4]);
- float promedioVentas(int m3[3][4]);

- 13) Se tienen dos matrices de 6 x 6, m1 y m2, se pide realizar las siguientes operaciones:

- void cargarMatriz (float matriz[6][6]);
- float sumarDiagonalPrincipal(float matriz[6][6]);
- float sumarDiagonalSecundaria(float matriz[6][6]);
- void sumarMatrices(float matriz[6][6], float matriz2[6][6], float matrizFinal[6][6]);
- float promedioMayoresA20(float matrizFinal[6][6]);



14) Se tienen dos matrices de 8 x 8, **m1** y **m2**, se pide realizar las siguientes operaciones:

- `void cargarMatriz (float matriz[8][8]);`
- `float sumarDiagonalPrincipal(float matriz[8][8]);`
- `void guardarDiagonalPrincipal(float matriz[8][8], float vector[8]);`
- `void sumarMatrices(float m1[8][8], float m2[8][8], float matrizFinal[8][8]);`
- `float promedio (float matrizFinal[8][8]);`
- `int cantidadMenoresANumero(float matrizFinal[8][8], float numero);`

15) Generar un programa que permita cargar una matriz de 4x4, dicha matriz estará llena de letras del abecedario. Las letras no pueden estar repetidas y se deben generar de forma aleatoria. Luego se pide mostrar la diagonal secundaria de la matriz.

El **main** debe verse así:

```
printf("----- ESTUDIANTE: xxxx ----- DNI: xxxxx ----- \n");

char matriz[TAM][TAM];
cargarMatriz(matriz);
mostrarMatriz(matriz);
mostrarDiagonalSecundaria(matriz);

cargarMatriz(matriz);
mostrarMatriz(matriz);
mostrarDiagonalSecundaria(matriz);
```

El resultado debe ser algo así:

```
-----ESTUDIANTE: xxxxxx----- DNI: nnnnnnn-----
e  u  k  s
i  l  r  a
c  n  t  y
d  b  w  h

--- s --- r --- n --- d ---

z  c  m  l
x  a  k  g
u  b  o  h
y  v  j  q

--- l --- k --- b --- y ---
```

16) Crear un procedimiento y llamarlo desde el **main**, en el cual se deberá imprimir por pantalla una pirámide, donde cada posición es un carácter. El procedimiento tiene definido el nombre y sus parámetros de entrada de la siguiente manera:

```
void imprimirPiramide(int tamano, char caracter);
```

Por ejemplo, si en el **main** se mandan como parámetro el 13 y la f, debe mostrarse por consola lo siguiente:

<pre> f fff fffff ffffffff ffffffff ffffffff ffffff ffffff</pre>	Observaciones Los espacios en blanco a la izquierda SON IMPORTANTES en el algoritmo. Si se envía como parámetro de entrada un número par, se lo debe cambiar por el impar anterior. Ejemplo: si se manda el 16 se tiene que trabajar con el 15. Los parámetros que se usan en el main son cualquiera, no hay que pedirlos al usuario, pongan cualquiera y dibujen como quedaría la pirámide.
---	---

- 17) Crear este algoritmo: void dibujarFigura (int filas, char carácter);

```
Numero de filas entre 10 y 30: 11
Ingrese un caracter: q
  q0
  qq1
  qqq2
  qqqq3
  qqqqq4
  qqqqqq5
  qqqqqqq6
  qqqqqqqq7
  qqqqqqqqq8
  qqqqqqqqqq9
  qqqqqqqqqqq10
```

- 18) Generar un programa que permita dibujar un triángulo, el procedimiento que lo dibuja recibe como parámetros la base del triángulo y una letra del abecedario. El triángulo deberá ser completado por las letras que continúan en el abecedario. Si se ingresa algo que no es una letra del abecedario se deberá mostrar un mensaje de error.

El main deberá verse así:

```
printf("----- DNI: xxxx ----- ESTUDIANTE: xxxxx ----- \n");

dibujarTriangulo(5, 'g');
dibujarTriangulo(8, 'p');
dibujarTriangulo(4, '$');
```

Resultado de la ejecución:

Notar que si el abecedario no alcanza se vuelve a empezar desde la a.

```
-----ESTUDIANTE: xxxxx----- DNI: nnnnnnnn-----
g
h  i
j  k  l
m  n  o  p
q  r  s  t  u

p
q  r
s  t  u
v  w  x  y
z  a  b  c  d
e  f  g  h  i  j
k  l  m  n  o  p  q
r  s  t  u  v  w  x  y

Error el caracter: $ no es una letra!!!
```

- 19) Crear un procedimiento que reciba el porcentaje redondeado de votos de 4 partidos políticos, y por medio de eso genere un gráfico de barras dibujado por un carácter cualquiera que se debe pedir por teclado. El procedimiento debe tener este aspecto y ser llamado desde el main.

```
void crearGraficoDeBarras(int n1, int n2, int n3, int n4, char c);
```

Debería mostrar algo así:

```
Partido1+++++++31%
Partido2+++++++18%
Partido3+++++++24%
Partido4+++++++10%
EnBlanco+++++++17%
```


- 20) Comentar todas las líneas del siguiente código fuente y decir qué es lo que se imprime por pantalla, además realizar la prueba de escritorio que justifique su respuesta.

```
1. int main(){
2. int * puntero1, * puntero2;
3. int vector[10];
4. int i = -3;
5. for (int k= 0; k<10; k++){
6. vector[k] = k +2*i;}
7. puntero1 = &vector[3];
8. puntero2 = &vector[6];
9. for (int k = 0; k<10; k++){
10. if(*puntero1 < *puntero2){
11. puntero1 = &vector[k];
12. vector[k] = vector[k]+3;
13. }}
14. printf("----- %d", *puntero1);
15. return 0;}
```

- 21) Comentar todas las líneas del siguiente código fuente y decir que es lo que se imprime por pantalla, además realizar la prueba de escritorio que justifique su respuesta.

```
1. int main(){
2. int x[3]={1,2,3};
3. int a = x[1];
4. int *puntero = &x[2];
5. for (int i = 0 ; i<3;i++){
6. x[i]= x[i]+ *puntero +a;
7. puntero = &x[i];}
8. for(int i = 0 ; i<3;i++){
9. printf("----> %d\n", *(puntero-i));}
10. return 0;
11. }
```

22) Realizar la prueba de escritorio y mostrar que imprime el siguiente código fuente:

<pre>#include <stdio.h> #include <stdlib.h> void funcion(int numero1, int numero2); int main(){ funcion(-2,3); return 0; }</pre>	<pre>void funcion(int numero1, int numero2){ int numero = numero1*numero2; if(numero<0){ numero = (-1)*numero; } int i,j,k; for(i=0;i<=numero;i++){ for(k=numero;k>i;k--){ printf(" "); } for(j=0;j<i+1;j++){ printf(" %d",j); } printf("\n"); } }</pre>
--	--

23) Crear un menú que tenga 4 opciones:

<pre>0) Salir 1) Mostrar decreciente 2) Mostrar creciente 3) Factorial, (utilizando ENUM).</pre>
--

Donde se deben mostrar los números de 0 a n , de n a 0 o el factorial de n según corresponda, utilizando funciones o procedimientos recursivos. El n debe ser ingresado por el usuario, y tiene que ser mayor que cero, de no ser así se debe ingresar otro valor.

24) Crear un menú, utilizando enumeradores, con el cual se pueda:

- 0) Leer un numero positivo o cero desde el archivo `numero.txt` y guardarlo en `int n`
- 1) Ingresar un número natural o cero (si se ingresa un número negativo el sistema debe pedir otro número), y guardarlo en `int n`
- 2) Pasar el número N de decimal a binario por medio de una función recursiva
- 3) Guardar el número binario en el archivo `numeroBinario.txt`
- 4) Salir del sistema

25) Crear un algoritmo recursivo, que transforme y escriba el número n , a una base b , b debe ser menor que 10 y mayor a 2. Se debe llamar a las 3 funciones o procedimiento por medio de un menú con enumeradores.

- `int pedirN();`
- `int pedirbase();`
- `void pasarYEscribirEnOtraBase (int n, int b); // Recursivo, escribe en numero.txt`

26) Utilizando una estructura llamada **Libro**, crear las funciones y procedimientos necesarios para cargar 10 libros (vector de **Libro**), mostrar los diez libros y ordenar los libros por año. El libro debe tener año, nombre y precio. Todo debe ser invocado desde el **main**.

27) Crear un TDA **Rectángulo**, completo, donde se tengan los atributos, altura, ancho, área y perímetro. Una vez creado, crear en el **main** 20 rectángulos pidiendo los datos por teclado, y mostrar el perímetro de los 20 rectángulos.

28) Crear un TDA **Círculo**, completo, en el cual el círculo tenga **float puntoX**, **float puntoY**, **float radio**, **float área** y **float perímetro**. Crear un menú con enumeradores, que permita,

0. Cantidad de círculos a crear (tamaño de array de círculos).
1. Crear círculo por teclado.
2. Mostrar datos del círculo.
3. Mostrar perímetro y área.

29) Crear un TDA completo, donde se puedan manejar los datos que caracterizan a un **Empleado**: **nombre**, **apellido**, **díasTrabajadosPorMes**, **sueldoDiario** y **sueldoTotalMes**. Una vez creado el TDA, crear en el **main** un array de 100 empleados creados por teclado, todos.

30) Se tiene un TDA **Detalle**, para trabajar los datos de los detalles de una factura. En **Detalle** tenemos, la descripción del producto, el precio unitario del producto, la cantidad vendida de dicho producto y el precio total que depende del unitario y de la cantidad comprada. Se pide crear un TDA completo (**.h**, **.c** y todo lo que implica un TDA completo) y crear dos detalles desde el **main**, uno por teclado y otro por parámetros.

31) Crear un TDA **Jugador**, donde se tengan los atributos, nombre, partidos, goles, promedioDeGol. El TDA debe estar completo tanto en los archivos **.h** como **.c**. Una vez creado el TDA crear en el **main** una estructura **Equipo**, que tenga un nombre y un array de 20 Jugadores. Crear el algoritmo que permita crear los 20 jugadores por teclado en el **main**.

32) Crear un TDA completo que permita trabajar con un “**Jugador**” de futbol, los datos relevantes del jugador son el nombre, la cantidad de goles, su edad, su habilidad (un float mayor que 0) y su valor. El valor de un jugador depende de sus goles, su edad y su habilidad.

Dicho valor en dólares es:

$$5 \text{ millones} * \text{habilidad} * (25/\text{edad}) + 1500 * \text{goles}$$

1. Crear 4 jugadores y guardarlos en un array.
2. Mostrar los jugadores.
3. Buscar dos jugadores, por nombre con un procedimiento.

El **main** deberá verse así, sin modificaciones:

```
printf("ESTUDIANTE: xxxxxx ---- DNI: xxxxxx \n\n");
//          nombres      goles edad habilidad
Jugador j1 = crearJugador("Juan M. Lucero", 11, 29, 0.21);
Jugador j2 = crearJugador("Lionel Messi", 753, 34, 2.66);
Jugador j3 = crearJugador("Kylian Mbappe", 165, 22, 2.87);
Jugador j4 = crearJugador("Crist. Ronaldo", 793, 36, 2.58);

Jugador plantel[4] = {j1,j2,j3,j4};

mostrarJugadores(plantel,4);          // Muestra los 4 jugadores
Jugador jugadorBuscado = crearJugador("Cristian Tarragona", 43, 29, 1.1);
buscarJugador(plantel, 4, jugadorBuscado); // Resultado de la búsqueda, proc con print
buscarJugador(plantel, 4, j2);          // Resultado de la búsqueda, proc con print
```

El resultado esperado de la ejecución debe ser así:

```
ESTUDIANTE: xxxxxxxx ---- DNI: nnnnnnnn

-----JUGADOR-----
NOMBRE: Juan M. Lucero
EDAD: 29
GOLES: 11
HABILIDAD: 0.21
VALOR: US$ 921672.38

-----JUGADOR-----
NOMBRE: Lionel Messi
EDAD: 34
GOLES: 753
HABILIDAD: 2.66
VALOR: US$ 10908912.00

-----JUGADOR-----
NOMBRE: Kylian Mbappe
EDAD: 22
GOLES: 165
HABILIDAD: 2.87
VALOR: US$ 16554317.00

-----JUGADOR-----
NOMBRE: Crist. Ronaldo
EDAD: 36
GOLES: 793
HABILIDAD: 2.58
VALOR: US$ 10147833.00
Jugador Cristian Tarragona NO ENCONTRADO
Jugador Lionel Messi Encontrado en la posicion: 1
```

33) Crear un TDA completo que permita trabajar con un **Elemento** de la tabla periódica, cada elemento deberá tener un número atómico (es lo mismo que la cantidad de protones), un símbolo químico, un nombre, un tipo, la cantidad de neutrones, un número másico (es la cantidad de protones más la cantidad de neutrones) y la cantidad de electrones (coincide con la cantidad de protones).

1. Crear 3 elementos.
2. Mostrar los 3 elementos.
3. Ordenar los 3 elementos por número másico, y mostrarlos.

El **main** deberá verse así, sin modificaciones:

```
int main(){
    printf("----ESTUDIANTE: xxxx ----- DNI: xxxxxx -----\n");

    Elemento e1 = crearElemento(25, "Mn", "Manganeso", "Transicion", 30);
    Elemento e2 = crearElemento(36, "Kp", "Kripton", "Gas noble", 50);
    Elemento e3 = crearElemento(11, "Na", "Sodio", "Metal alcalino", 12);

    Elemento tabla[3] = {e1,e2,e3};

    mostrarElementos(tabla, 3);

    ordenarElementos(tabla, 3);
    printf("\nOrdenados\n");
    mostrarElementos(tabla, 3);

    return 0;
}
```

El resultado esperado de la ejecución debe verse así:

```
-----ESTUDIANTE: NICO --- DNI: 11111-----
55          <---Numero masico
Mn          <---Numero Simbolo
25          <---Numero Atomico

Ese es el elemento llamado Manganeso
PROTONES: 25
ELECTRONES: 25
NEUTRONES: 30
Y es del tipo: Transicion
-----
86          <---Numero masico
Kp          <---Numero Simbolo
36          <---Numero Atomico

Ese es el elemento llamado Kripton
PROTONES: 36
ELECTRONES: 36
NEUTRONES: 50
Y es del tipo: Gas Noble
-----
23          <---Numero masico
Na          <---Numero Simbolo
11          <---Numero Atomico

Ese es el elemento llamado Sodio
PROTONES: 11
ELECTRONES: 11
NEUTRONES: 12
Y es del tipo: Metal Alcalino
-----
Ordenados
-----
23          <---Numero masico
Na          <---Numero Simbolo
11          <---Numero Atomico

Ese es el elemento llamado Sodio
PROTONES: 11
ELECTRONES: 11
NEUTRONES: 12
Y es del tipo: Metal Alcalino
-----
55          <---Numero masico
Mn          <---Numero Simbolo
25          <---Numero Atomico

Ese es el elemento llamado Manganeso
PROTONES: 25
```

- 34)** Crear dos TDA anidados (no hace falta que estén completos con get, set pre y pos, solo lo que se pide y lo que necesiten para realizar correctamente TDA), que permitan cargar y mostrar los datos de **Personas** y de sus datos **Nacimiento**. Del nacimiento nos interesa saber el día, mes, año y ciudad de nacimiento. De la persona solamente nos interesa saber su nombre y los datos del nacimiento. Con dichos datos se puede calcular el signo del horóscopo chino de la persona, este signo es un atributo de la persona y para simplificar el ejercicio se va a definir que es de dragón si el resto de la división por 12 del año de nacimiento es 8, de otra forma se dirá que es de otro signo.

El **main** deberá verse así, sin ninguna modificación (salvo sus datos personales):

```
int main(){
    printf("----- DNI: xxxx ----- ESTUDIANTE: xxxxx ----- \n");

    Persona p1 = crearPersona("Romina", 12, 3, 1988, "Sevilla , E.");
    mostrarPersona(p1);

    Persona p2 = crearPersona("Nykolai", 9, 1, 2001, "Kiev, U.");
    mostrarPersona(p2);

    return 0;
}
```

El resultado debe verse así:

```
-----DNI: nnnnnnn ----- ESTUDIANTE: xxxxxxxxxx -----

-----PERSONA-----
NOMBRE: Romina
SIGNO: Dragon

-----NACIDO-----
FECHA: 12/3/1988
EN: Sevilla, E.

-----PERSONA-----
NOMBRE: Nykolai
SIGNO: Otro

-----NACIDO-----
FECHA: 9/1/2001
EN: Kiev, U.
```

- 35)** Crear dos TDA anidados (no hace falta que estén completos con get, set pre y pos, solo lo que se pide), que permitan cargar y mostrar los datos de **Bandas** de música y de sus **Lideres** musicales.

De la banda sabemos su nombre, su género, año de fundación, su sitio web y su canción más popular. Además, sabemos que cada banda tiene un líder, de ese líder nos interesa su nombre completo, su año de nacimiento, su ciudad natal y su estado a la fecha (vivo, muerto, etc).

El constructor de la banda será por parámetros mientras que los datos del líder se tienen que ingresar por teclado.

El **main** deberá verse así, sin ninguna modificación (salvo sus datos personales):

```
int main(){
    printf("----- DNI: xxxx ----- ESTUDIANTE: xxxxx ----- \n");

    Banda b1 = crearBanda("Nirvana", "Grunge", 1987, "nirvana.com", "Smells Like Teen Spirit");
    mostrarBanda(b1);

    Banda b2 = crearBanda("The Beatles", "Rock", 1962, "thebeatles.com", "Come Together");
    mostrarBanda(b2);

    return 0;
}
```

El resultado de la ejecución deberá ser exactamente así (imagen partida en dos partes para que no quede tan extensa):

-----ESTUDIANTE: XXXXXX ----- DNI: NNNNNNN Cargando lider de la banda: Nirvana Por favor ingresar su nombre: Kurt Cobain Por favor ingresar su ciudad natal: Aberdeen Por favor ingresar su estado Muerte por arma de fuego Por favor su año de nacimiento 1967 -----BANDA----- NOMBRE: Nirvana GENERO: Grunge FUNDADA: 1987 SITIO: nirvana.com POPULAR: Smells Like Tenn Spirit -----LIDER----- NOMBRE: Kurt Cobain NACIDO: 1967 CIUDAD: Aberdeen ESTADO: Muerte por arma de fuego	Cargando lider de la banda: The Beatles Por favor ingresar su nombre: John Lennon Por favor ingresar su ciudad natal: Liverpool Por favor ingresar su estado Asesinado Por favor su año de nacimiento 1949 -----BANDA----- NOMBRE: The Beatles GENERO: Rock FUNDADA: 1962 SITIO: thebeatles.com POPULAR: Come Together -----LIDER----- NOMBRE: John Lennon NACIDO: 1949 CIUDAD: Liverpool ESTADO: Asesinado
--	---

36) Crear un programa que permita administrar el Mundial de Qatar y sus **Estadios**. De los estadios sabemos su nombre, su ciudad y su capacidad, además sabemos que los estadios son 8 como máximo. Mientras que del **Mundial** sabemos su año, el país y el nombre de la mascota. Se pide que el programa haga lo siguiente:

1. Crear el mundial con los siguientes datos: **año = 2022, país = Qatar, mascota = Laeeb** y mostrarlo por pantalla, los estadios deben estar vacíos inicialmente.
2. Agregar 4 estadios por parámetros, nombre, ciudad y capacidad: (Bin Hamad, Doha, 13000), (Khalifa, Doha, 48000), (Ahmad Bin Ali, Doha, 45000), (Al Janoub, Al Wakrah, 40000). Y mostrar el mundial.
3. Crear una función que permita buscar estadios por ciudad y capacidad. Probar la función buscando un estadio en la ciudad de Doha con 45000 de capacidad.
4. Crear un procedimiento que guarde toda la información en un archivo utilizando la técnica de separación por delimitadores.

37) Crear un programa que permita administrar los datos de un **Museo**, del museo sabemos el nombre y la dirección, además de conocer a su **Director**, del cual conocemos su nombre, apellido y dni. El museo tiene un máximo de 10 **Obras** de arte, de las cuales conocemos su nombre, autor y precio. El programa deberá anidar correctamente las 3 estructuras solicitadas.

1.
 - Crear por parámetros el museo: Histórico Nacional, Defensa 1600
 - Cuyo director es: Gabriel, Di Meglio, 11222333
 - Inicialmente las 10 obras deben estar vacías.
 - Mostrar el museo con todos sus datos
2. Insertar por parámetro las siguientes 3 obras
 - Noche estrellada, Van Gogh, 23 millones
 - El Beso, Gustav Klimt, 17 millones
 - El Grito, Edvard Munch, 11 millones
 - Mostrar el museo con todos sus datos
3. Ordenar las obras por nombre y mostrar el museo con todos sus datos.

38) Se tiene el archivo `ventas.txt`, con la estructura:

```
1989;120003
1990;233100
.
.
2019;111022
```

Donde se almacenan los años y cantidad de unidades vendidas de algún producto. Se pide cargar estos valores en un array de la Struct pertinente. Una vez cargado el array, realizar la búsqueda secuencial para encontrar algún año, y mostrar por pantalla el promedio de ventas por año.

39) Se tiene un archivo `vehiculos.txt` donde se almacenan los modelos de los rodados y las patentes ya ordenados por modelos y delimitados por el carácter `#`. El archivo tiene este aspecto:

```
Alfa Romeo 145#RBC 587
Audi TT#AB 123 CC
.
.
Volvo 34#ABT 331
```

Crear un programa que lea todos estos registros y los almacene en la estructura o array pertinente.

Una vez cargados en memoria todos estos registros, realizar una búsqueda secuencial que permita buscar **Vehículos** por nombre y patente, si el vehículo existe se debe retornar la posición del array donde se lo encontró, en caso de que no exista se debe retornar -1.

40) Se tiene el archivo `nombres.txt`, con nombres uno debajo del otro. Crear un array de una struct que posea solo la variable nombre, el array debe tener tantas posiciones como renglones hay en el archivo. Una vez cargado el array ordenarlo por nombre y escribir los nombres ordenados en un archivo `ordenados.txt`.

41) Se tiene el archivo `numeros1.txt`, con números enteros ordenados uno debajo del otro. Se pide crear un array con estos números, el array debe tener la dimensión exacta de números que había en el archivo. Una vez creado el array crear la búsqueda binaria que nos permita saber si está el número `n`. Crear un procedimiento que escriba en un archivo `numeros2.txt`, con el mismo formato, los números pares del array. Todo esto debe estar disponible por medio de un menú con enumeradores. Las funciones y procedimientos que deben estar si o si son las siguientes:

- `int leerTamanoArchivo();`
- `void cargarArchivoAlArray(int array[], int tamano);`
- `int busquedaBinaria(int array[], int tamano, int *busco);`
- `void escribirPares(int array[], tamano);`